Table of Contents

1. Introduction 2
2. Performance evaluation 2
2.1 Accuracy 2
2.2 Precision 2
2.3 Recall 2
2.4 F1-score 2
2.5 AUC-ROC 2
3.Dataset Experimental Setup 3
3.1 Training Data: 3
3.2 Testing Data: 3
4.1 Logistic Regression 3
4.1.1:Data Preprocessing (baseline model) 3
4.1.2: Features filtering 3
4.1.3: Test Model Complexity By C (most better) 3
4.2 RandomForest 4
4.2.1: Test Model Complexity By D 4
4.3 XGBoost 4
4.3.1: Gradient Boosting 4
6. Overfitting Challenges 6
6.1 Number of Dataset 6
6.2 Tuning Strategies 6
6.3 Validation Limitations 6
7. Result Analysis 6
8. Reference 7
9. Appendix ..... 錯誤! 尚未定義書籤。

Kaggle Don't Overfit! II Competition: A Study on Avoiding Overfitting in Machine Learning

1. Introduction

Don't Overfit! II competition, which hosted on Kaggle and it is a machine learning challenge designed to test participants' ability to build models that generalize well on a small, high-dimensional dataset. Overfitting, where a model learns noise and specific patterns in the training data that do not apply to unseen data, is a central problem in machine learning (mutasa s. et al, 2020). This competition emphasizes techniques to combat overfitting, such as regularization, feature selection, and careful model selection, given a training dataset with only 250 samples and a test set of 19,750 samples. The evaluation metric is the Area Under the Receiver Operating Characteristic Curve (AUC-ROC), which measures the model's ability to distinguish between binary classes 0 and 1.

As a beginner in machine learning, my goal was to learn about overfitting and apply supervised learning models to achieve competitive AUC-ROC scores. I explored three algorithms, such as Logistic Regression, Random Forest, and XGBoost. Logistic Regression implementing multiple versions with increasing complexity, including feature scaling, feature selection, and hyperparameter tuning. This report documents my experimental setup, model implementations, and performance, which will include the methods in terms of accuracy, precision, recall, F1 score and AUC-ROC, providing insights into overfitting mitigation strategies and their impact on model generalization.

2. Performance evaluation

This report importing five important indicators were introduced in the study, each of which corresponds to the actual results of the algorithm. These metrics include accuracy, precision, recall, F1-score and AUC-ROC, and these performance evaluation metrics are as follows:

2.1 Accuracy: The ratio of correctly predicted instances to the total.

$$\text{Accuracy} = \frac{True\ Positives + True\ Negatives}{Total\ Instances}$$

2.2 Precision: The ratio of correctly predicted positive observations to the total predicted positives.

$$\text{Precision} = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

2.3 Recall: The ratio of correctly predicted instances to the total.

$$\text{Recall} = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

2.4 F1-score: The ratio of correctly predicted instances to the total.

$$\text{F1} - \text{score} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

2.5 AUC-ROC: The ROC curve is a plot of the True Positive Rate (TPR) versus the False Positive Rate (FPR)

$$\text{TPR} = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

$$\text{FPR} = \frac{False\ Positives}{False\ Positives + False\ Negatives}$$

3.Dataset Experimental Setup

The dataset for Don't Overfit! II competition, specifically the "Older Dataset for Don't Overfit II Challenge" provided by user Md Mubasir on Kaggle, consists of two CSV files:

3.1 Training Data:

train.csv: Contains 250 rows and 302 columns, including an 'id' column, a binary 'target' column , and 300 numerical feature columns labeled 0 to 299

3.2 Testing Data:

test.csv: Contains 19,750 rows and 301 columns, including an 'id' column and the same 300 feature columns, but no 'target' column, as it is used for Kaggle's evaluation.

The small training set size relative to the number of features (250 rows vs. 300 features) poses a significant risk of overfitting, making preprocessing and model simplicity critical. The experimental setup was implemented in a Python Jupyter Notebook on Kaggle, using the sklearn and XGBoost libraries. The setup involved the following preprocessing and experimental steps:

4.1 Logistic Regression

4.1.1:Data Preprocessing (baseline model)

Version_1 submission (Private Score:0.698), the training data was loaded, and the 'id' and 'target' columns were dropped to create a feature matrix for x_train_data and y_train_data, and no scaling or feature selection was applied, serving as a reference for subsequent improvements. This version uses the default parameters (C=1.0, no feature preprocessing).

4.1.2: Features filtering

Version_2 submission (Private Score:0.696), which extended Version_1 and enhanced with feature standardization. Which is because on smaller dataset, the useless features will reduce the model's performance (Demšar, J. et al, 2021). So, to normalize feature scales and feature selection, set threshold = 0.7 to remove low variance features, reducing overfitting risk and the AUC-ROC result of V2 is better with the V1 (0.751736 - 0.743056).

4.1.3: Test Model Complexity By C (most better)

Version_3 submission (Private Score:0.707), which extended Version_2 by tuning the regularization parameter C values. Since the training dataset only included 250 rows of data, by testing the model complexity impact may improve the overfitting problem (Lever, J. et al, 2016). To test C values with [0.005, 0.01, 0.03, 0.05, 0.1, 0.5, 1.0, 1.5, 3.5, 5.5, 7.5, 10.0]. The highest validation AUC and that C value was selected. Finally, the selected best C will improve the model complexity to fit this training data. In this dataset, Best C=0.005, Best AUC=0.7743 and Public Score = 0.746

4.1.4: Define C with precision & recall

Version_4 submission (Private Score:0.696), on Version_3, which noticed that to change the model complexity with C can improve the overfitting problem. However, Version_4's accuracy was 0.66, lower than Version_2. Therefore, the goal of Version_4 was to optimize Version_3 by minimizing the absolute difference between precision and recall, thereby achieving a balanced performance.

Tabel 1: Logistic Regression V1 – V4 [Data taken from "dsai4203-ml-22027226d.ipynb" of the result graphs]

Version	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Private Score
V1	0.720000	0.750000	0.843750	0.794118	0.743056	0.698
V2	0.720000	0.750000	0.843750	0.794118	0.751736	0.696
V3	0.660000	0.666667	0.937500	0.779221	0.774306	0.707
V4	0.720000	0.750000	0.843750	0.794118	0.751736	0.696

4.2 RandomForest

4.2.1: Test Model Complexity By D

RandomForest submission (Private Score:0.590), which redefined the Version_3 of the Logistic Regression (LR). On LR_V3 and LR_V4, which are displayed by minimizing the absolute difference between precision and recall which is still causing the overfitting. It means that by using the AUC-ROC to fine the D value to reduce the model complexity. Which can avoid overfitting.

Its submission is ready computed the data preprocessing and features filtering, since which is based on the LR_V3 to redefine the logic to find the best D. Which RandomForest are used scikit learns Random Forest Classifier with the n_estimators=10 and tuned max_depth over [1, 2, 3, 4, 5, 6, 7, 8, 10] to control tree complexity. Finally, best_D=2, best_auc=0.6675

RandomForest is an ensemble method that builds multiple decision trees and averages their predictions, reducing variance and overfitting compared to single trees. However, when dataset is too small, stratified sampling may result in too few samples in each stratum, thereby increasing the risk of the model overfitting on the training set (Demšar, J. et al, 2021).

4.3 XGBoost

4.3.1: Gradient Boosting

XGBoost is a gradient boosting algorithm (Private Score:0.766, the best one), that builds sequential trees, optimizing a loss function with regularization to prevent overfitting. It is powerful for small datasets. So, in case on this 250 rows dataset is so porwerful here.

This XGB Classifier applied the n_estimators=100, learning_rate = 0 and max_depth set to the best D value from RandomForest tuning. Additional parameters included use_label_encoder=False, eval_metric='logloss'

Due to the simplicity and limited capacity of each tree, each step adds only a small and manageable amount of complexity. Multiple weak learners can be combined to model complex patterns without overfitting a single learner. Furthermore, although this project does not utilize the training stop function, as a small training dataset can lead to degraded model performance. But XGB was also provided with a function for prematurely stopping training after a specified number of epochs, effectively preventing overfitting.

Tabel 2: Best 3 model (Logistic Regression (LR), RandomForest (RF) and XGBoost (XGB))

[Data taken from "dsai4203-ml-22027226d.ipynb" of the result graphs]

Version	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Private Score
LR_V3	0.660000	0.666667	0.937500	0.779221	0.774306	0.707
RF	0.620000	0.632653	0.968750	0.765432	0.667535	0.590
XGB	0.680000	0.722222	0.812500	0.764706	0.713542	0.766

5. Result & Team Info

XGBoost> Logistic Regression >RandomForest

submission_v6_XGBoost (1).csv Complete (after deadline) · 3m to go	0.766	0.796
submission_v5_RandomForest (1).csv Complete (after deadline) · 3m to go	0.590	0.603
submission_v4_Cindex0.csv Complete (after deadline) · 2m to go	0.696	0.737
submissionv_v3 (1).csv Complete (after deadline) · 2m to go	0.707	0.746
submissionv_v2 (1).csv Complete (after deadline) · 2m to go	0.696	0.737
submission_v1 (1).csv Complete (after deadline) · 2m to go	0.698	0.740

Logistic Regression 0.707

RandomForest 0.590

XGBoost 0.766 🏆

Don't Overfit! II

A Fistful of Samples

Overview Data Code Models Discussion Leaderboard Rules Team Submissions

Your Team

Everyone that competes in a Competiton does so as a team - even if you're competing by yourself. Learn more.

General

TEAM NAME

yeung zero

This name will appear on your team's leaderboard position.

Let others know you're looking for teammates

Forming teams closed on May 8, 2019.

Team Members

yeung zero (You)

Team Leader

Save Changes

6. Overfitting Challenges

6.1 Number of Dataset:

The small training set (250 rows) and high feature count made overfitting a constant risk. Models like Random Forest showed sharp AUC drops at higher depths, illustrating how complex models can memorize noise. Regularization (e.g., low C in Logistic Regression, low max_depth in RandomForest/XGBoost) was critical to maintain generalization.

6.2 Tuning Strategies:

AUC-based tuning (Logistic Regression V3) outperformed precision-recall balancing (Logistic Regression V4), emphasizing the need to align tuning with the evaluation metric (AUC-ROC), it can also use the VarianceThreshold reduced noise (Feature filtering), improving validation AUC, like Logistic Regression of V2.

6.3 Validation Limitations:

Linear models (Logistic Regression) were robust but limited in capturing non-linear patterns, while tree-based models (Random Forest, XGBoost) risked overfitting but offered higher potential when tuned properly. The small validation set led to noisy AUC estimates (e.g., XGBoosts validation AUC=0.7135 vs. private score =0.766), suggesting cross-validation for future experiments.

7. Result Analysis

Logistic Regression Version 1(Baseline Model): I began with a basic Logistic Regression model without preprocessing, using default parameters (C=1.0). This served as a starting point to understand the datasets challenges. The validation AUC (~0.7431) were promising, but the private score (0.698) indicated potential overfitting, as the model might have captured noise in the training data. This experience highlighted the need for preprocessing to stabilize performance.

Logistic Regression Version 2(Adding Preprocessing): To reduce overfitting, I introduced feature standardization (StandardScaler) and feature filtering. Standardization ensured features had comparable scales, critical for Logistic Regression. The validation AUC improved slightly (~0.7431 up to ~0.7517), but the private score (0.696) was similar to Version 1(0.698). Which feature selection helped AUC, but it may still cause the overfitting. This step noticed me that preprocessing is essential for small datasets to filter noise, and the model should test the complexity to reduct the overfitting.

Logistic Regression Version 3(Hyperparameter Tuning for Regularization): Recognizing that regularization could further combat overfitting, I tuned the regularization parameter C value over a range [0.005, 0.01, 0.03, 0.05, 0.1, 0.5, 1.0, 1.5, 3.5, 5.5, 7.5, 10.0], selecting the value with the highest validation AUC. The best C=0.005 yielded a validation AUC (~0.7743) and a private score (0.707), the highest among Logistic Regression versions. This improvement confirmed that stronger regularization (lower C) prevented the model from overfitting by penalizing large weights, a key lesson in managing model complexity.

Logistic Regression Version 4 (Balancing Precision and Recall): To explore alternative model selection criteria, I modified Version 3 to choose C based on minimizing the absolute difference between precision and recall percentages, aiming for balanced performance. However, this reverted to a validation AUC (~0.7517), which is lower with Version 3 and private score is reduced to 0.696, indicating that AUC-based tuning (Version 3) was more effective for the competitions to avoid overfitting. This experiment taught me that prioritizing secondary metrics precision & recall over AUC-ROC could misalign with the competitions.

RandomForest (Exploring Non-Linear Models): To capture potential nonlinear patterns, implement a Random Forest Classifier, tuning max_depth over [1, 2, 3, 4, 5, 6, 7, 8, 10] with n_estimators=10. The best max_depth=2 gave a validation AUC of ~0.6675, but the private score was poor (0.590). Higher depths (e.g., AUC~0.4523 at max_depth=10) showed severe overfitting, as trees memorized training noise. The low n_estimators likely caused instability, teaching me that Random Forests require sufficient trees and careful depth control to generalize on small datasets.

XGBoost (Gradient Boosting): Finally, by testing XGBoost, a gradient boosting algorithm, which model was finded on the forum and which is the most recommended and most used model on the Kaggle. The improvement over Random Forest suggested that boosting better captured complex patterns while its built-in regularization (eval_metric='logloss') mitigated overfitting. It applied the best max_depth=2 (definded on RandomForest_Ver), with n_estimators=100 and learning_rate=0.1. This achieved a validation AUC of ~0.7135 and the highest private score (0.766).

8. Reference

Mutasa, S., Sun, S., & Ha, R. (2020). Understanding artificial intelligence based radiology studies: What is overfitting? *Clinical Imaging*, *65*, 96–99. https://doi.org/10.1016/j.clinimag.2020.04.025

Demšar, J., & Zupan, B. (2021). Hands-on training about overfitting. *PLoS Computational Biology*, *17*(3), e1008671. https://doi.org/10.1371/journal.pcbi.1008671

Lever, J., Krzywinski, M., & Altman, N. (2016). Points of Significance: Model selection and overfitting. *Nature Methods*, *13*(9), 703. https://doi.org/10.1038/nmeth.3968